Linear Regression via Gradient Descent

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Reframe fitting a linear model as **training**: iteratively adapting parameters using a loss, not solving for them in one shot
- Explain why we use an iterative approach here even though linear regression has a closed-form solution
- Compare full-batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, and describe the trade-offs between them

The problem, again

What we already have

- A dataset of examples: pairs $(\mathbf{x}, y)$
- A model with parameters we can tune:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$$

- A way to score any choice of $(\mathbf{w}, b)$
- All three pieces came from last lecture

So what's actually missing?

- Last time, we picked `(w, b)` by hand, or let a random walk stumble into something decent
- A real dataset can have thousands of points and dozens of features — eyeballing it, or getting lucky, doesn't scale
  - As we add more dimensions we need more weights — `w` gets bigger, and a random step is decreasingly likely to land in a direction that actually helps
- We need a **systematic, automatic** way to adapt `(w, b)` toward a lower loss

That process — adapting parameters using data and a loss — is what **training** a model means.

How do we quantify how well we’re doing?

Loss functions

- We need a single number that says “how bad is this model, right now?”
- That number is called a **loss** — smaller is better
- For regression, we already have one: mean squared error

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}_i - y_i \right)^2, \qquad \hat{y}_i = \mathbf{w} \cdot \mathbf{x_i} + b$$

Every method we build today — random search, gradient descent, SGD, mini-batch — is really just a different answer to: **how do we use the loss to pick the next `(w, b)`?**

Why not just solve it directly?

- Linear regression actually *has* a closed-form solution — the normal equation:

$$\mathbf{w}^\star = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- So why not just… do that?

> **Note**
>
> This folds `b` into `w` by adding a column of 1s to `X` — a standard convention, but new notation if you haven't seen it before.

1. That matrix inverse gets expensive fast as the number of features grows
2. It only exists *because* the model is linear — there's no equivalent formula once the model isn't
3. Every model we build for the rest of this course — including ones with no closed form at all — gets fit the *same* iterative way
4. Better to learn the general mechanism now, on a model simple enough to see clearly

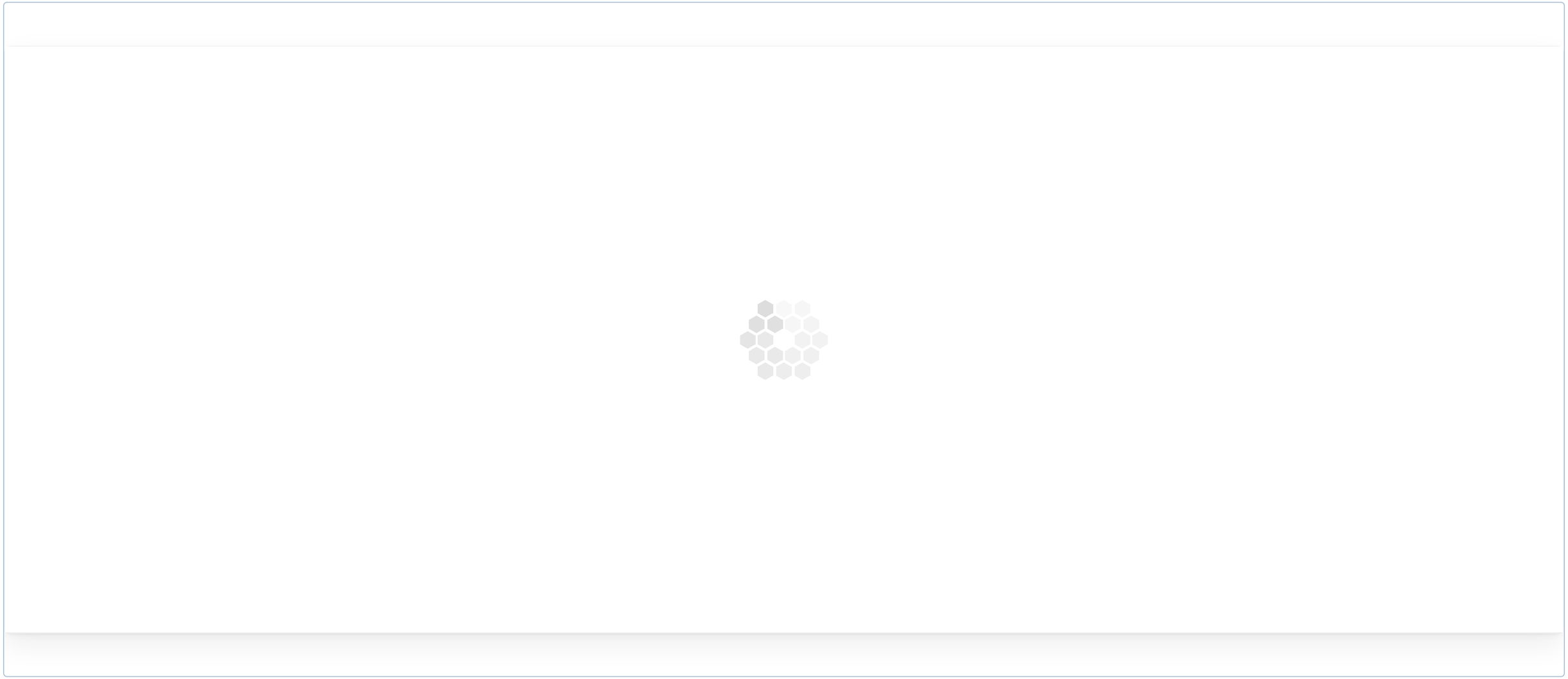
Four ways to adapt `(w, b)`

We'll watch the same dataset get fit four different ways:

1. **Random search** — nudge and hope (already met this one)
2. **Gradient descent** — use *all* the data to take one confident step
3. **Stochastic gradient descent** — use just *one* point per step
4. **Mini-batch gradient descent** — a tunable middle ground

Random search, again

Random search, live



Important

We’re scoring every guess against all **40** data points here — each “better or worse” call means running the whole dataset through the model

Still no sense of direction

- Same conclusion as before: blue = improved, red = made it worse — no way to tell in advance which a step will be
- With noisier data, the random walk struggles even more — more room to wander before stumbling onto a decent fit
- This is our baseline for the rest of the lecture — everything from here on should beat it

Remember, each time we say “better or worse” we need to do calculations!

Gradient Descent

Train using all the data

The idea, before the math

> Each step, decrease the “badness” of the model as fast as possible

- “Badness” is just our loss — so the goal is to **minimize the loss**
- Random search nudged `(w, b)` and *hoped* that helped
- Gradient descent instead asks: of every direction we could move `(w, b)`, which one decreases the loss *fastest right now* — and moves exactly that way

The loss, written out

- Same MSE loss as before, now written as a function of the *whole* dataset:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}_i - y_i \right)^2, \qquad \hat{y}_i = w x_i + b$$

x_i, y_i	one training example	constant — fixed by the dataset
N	number of training examples	constant
w, b	model parameters	**trainable** — what we're actually adjusting
$\hat{y}_i$	prediction	derived — changes whenever w, b do

Given this measure of how bad we are, what do we do?

The gradient of the loss

- Take the partial derivative of L with respect to each *trainable* parameter (w and b):

$$\frac{\partial L}{\partial w} = \frac{2}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)\,x_i \qquad \frac{\partial L}{\partial b} = \frac{2}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)$$

- Stack them into a gradient vector, same idea as ∇f from last lecture:

$$\nabla L = \left[\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}\right]$$

- This points toward *steepest loss* — so we step in the **opposite** direction

The gradient descent algorithm

1. Compute the prediction $\hat{y}_i$ for every one of the N points, using the current (w, b)
2. Compute the gradient ∇L from those predictions — average the per-point contributions
3. Update: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$, and $b \leftarrow b - \eta \frac{\partial L}{\partial b}$
4. Repeat until the loss stops improving (or some other criteria – future work)

η (“eta”) is the **learning rate** — how big a step to take in the gradient’s direction

Aside: What could go wrong with η?

- Too small, and training crawls — technically correct, painfully slow
- Too large, and a step can overshoot the bottom of the bowl entirely, making the loss *worse* — try pushing it too far in the widget below and see for yourself
- Choosing η well is its own skill; we'll come back to it later in the course

Gradient descent, live

- Same 40 points, same noise as before — but now every step uses the **exact** gradient from all of them, not a guess
- Blue = the step improved the loss, red = it made things worse — watch what happens as you raise η

That’s a very different walk

- Every early step moves confidently downhill — no wasted red steps, unlike random search
- …unless η was too large, in which case *every* step turns red and the loss explodes — that’s η doing exactly what we warned about



Caution

The catch: every single step required looking at **all 40** points. Scale that up to a real dataset, and each step gets expensive fast

Stochastic Gradient Descent

Train using one point at a time

So do we need to evaluate on everything?

- Every gradient descent step needs the error from **all 40** points, every time
- What if we didn’t? What if we picked **one point** and stepped using just its gradient?
  - We’ll eventually see all the data anyway, right?
- That’s the idea behind **stochastic gradient descent (SGD)** — “stochastic” because which point we grab is random

Why would this be good?

- Each step is now cheap and constant-time — it doesn't matter whether the dataset has 40 rows or 40 billion
- We can take *far* more steps for the same amount of computation
- We never need the whole dataset in memory at once — it can even arrive one example at a time, as a stream

The Catch

- One point's gradient is *not* the loss's true gradient — it's a cheap, noisy estimate of it
- A single unusual point can point the update in a bad direction entirely
- Expect the path to visibly **bounce**, not glide smoothly downhill the way full-batch gradient descent did

The SGD update

- The full-batch gradient averaged the per-point contributions:

$$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)\, x_i$$

- SGD just uses **one term of that sum** — no averaging, no waiting on the rest of the data:

$$\frac{\partial L_i}{\partial w} = 2(\hat{y}_i - y_i)\, x_i, \qquad \frac{\partial L_i}{\partial b} = 2(\hat{y}_i - y_i)$$

- Same formula, same update rule — just computed from one example instead of all of them

The stochastic gradient descent algorithm

1. Pick one training example (x_i, y_i) at random
2. Compute the gradient of the loss using *just that point*
3. Update (w, b) the same way as before: step opposite the gradient, scaled by η
4. Repeat — a different random point every time

SGD, live

- Same 40 points, same noise, same starting point as gradient descent
- Each step looks at just **one** highlighted point — watch how much more it bounces around

That’s a different kind of walk entirely

- No two steps move the same way — direction depends entirely on which point got picked
- It’s noisy, but it isn’t going nowhere — over enough steps, it still drifts toward the loss’s minimum, on average

> **Tip**
>
> The catch: we traded “confident but expensive” for “cheap but jumpy”.
>
> What’s the middle ground?

Mini-batch Gradient Descent

Train using a handful of points at a time

The idea

- SGD went too far one way — one point is barely a hint of the true gradient
- Full gradient descent went too far the other way — every step costs the whole dataset
- **Mini-batch gradient descent**: average the gradient over a small, randomly-sampled **batch** of k points — pick anywhere on the dial between the two

The mini-batch update

- Same gradient formula as full-batch, just averaged over `k` points instead of all `N`:

$$\frac{\partial L}{\partial w} \approx \frac{2}{k} \sum_{i \in \text{batch}} (\hat{y}_i - y_i)\, x_i \qquad \frac{\partial L}{\partial b} \approx \frac{2}{k} \sum_{i \in \text{batch}} (\hat{y}_i - y_i)$$

> **Important**
>
> SGD and Gradient Descent are **special cases** of mini-batch Gradient Descent

- `k = 1` *is* SGD
- `k = N` *is* full-batch gradient descent

The mini-batch gradient descent algorithm

1. Sample a random batch of k points (without replacement) from the dataset
2. Compute the gradient of the loss using just that batch
3. Update (w, b) the same way as before: step opposite the gradient, scaled by η
4. Repeat — a different random batch every time

Mini-batch, live

- Same 40 points, same noise as every widget today — now you pick the batch size k yourself
- The k sampled points light up in orange each step — watch the walk get smoother as k grows

What did we learn by turning the dial?

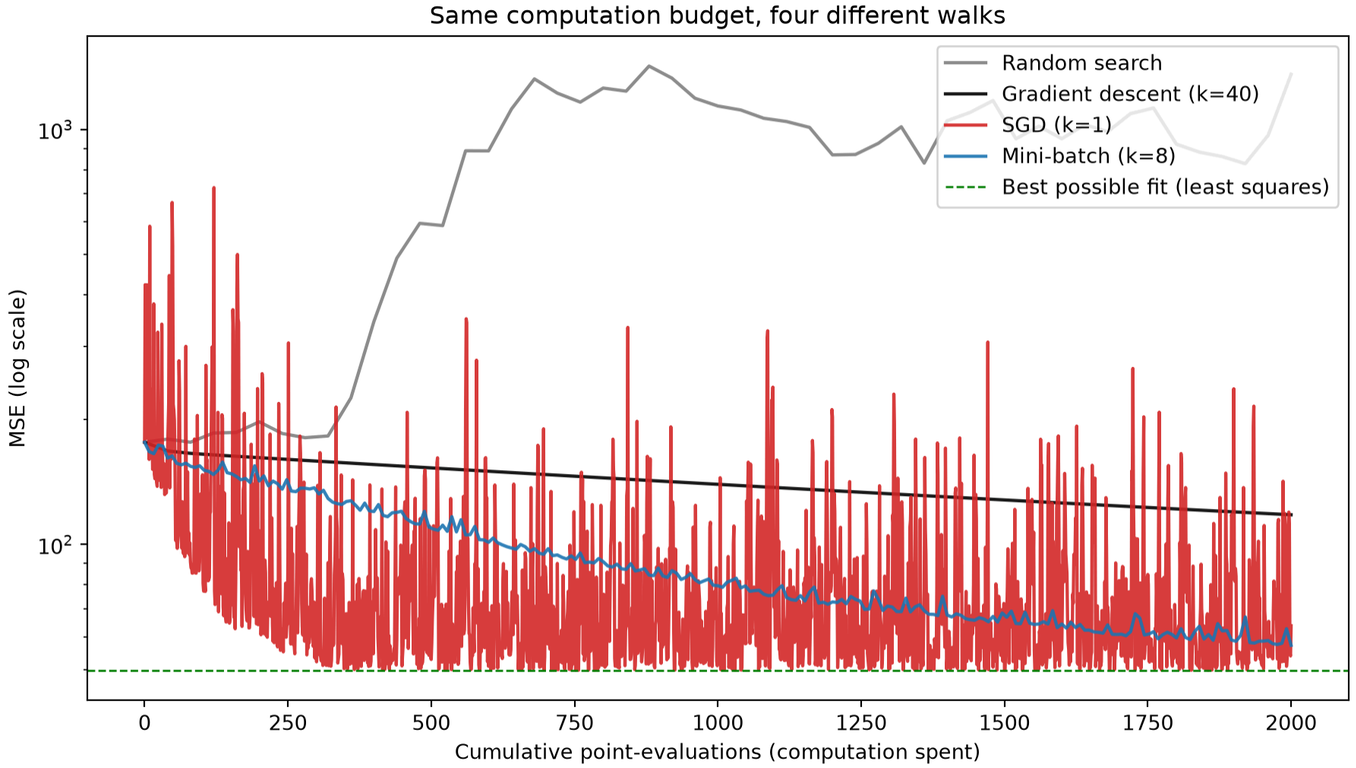
- Small k: cheap steps, lots of them, but a bouncy path — SGD's territory
- Large k: confident steps, but each one costs more — gradient descent's territory

- In practice, k gets chosen to balance noise against how much data fits comfortably in memory at once — commonly powers of 2, like 32, 64, 128
- This dial is what real training loops actually use — "SGD" in PyTorch almost always means *mini-batch* SGD under the hood

Comparing the four

Same computation budget, four different walks

- All four methods below get the *exact same* computation budget: 2,000 point-evaluations
  - They all still have the 40 point dataset so this is 50 **epochs**
    - An **epoch** is one full pass through the dataset
- Notice the difference in how the evaluation budget gets spent


Same computation budget, four different walks
Random search
Gradient descent (k=40)
SGD (k=1)
Mini-batch (k=8)
Best possible fit (least squares)
MSE (log scale)
10^3
10^2
0
250
500
750
1000
1250
1500
1750
2000
Cumulative point-evaluations (computation spent)

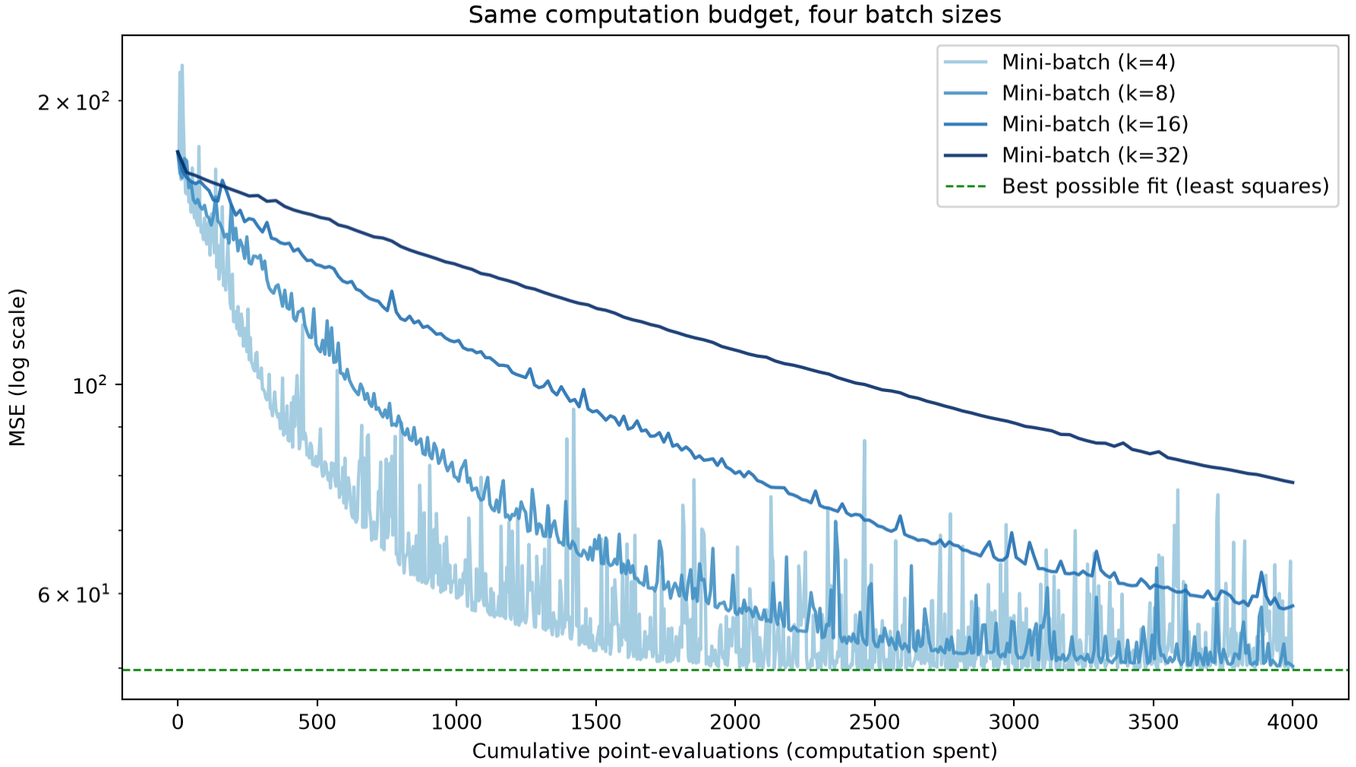
What the plot is telling us

- **Gradient descent** (black) descends smoothly and predictably — but for this budget, it only gets 50 steps (50 epochs, one step per epoch)
- **SGD** (red) gets 2,000 steps for the same budget — noisy, but it's still the fastest to a low loss here
- **Mini-batch** (blue) splits the difference — smoother than SGD, far more steps than GD
- **Random search** (gray) never gets a sense of direction — with nothing pulling it back, the loss can end up *anywhere*, and typically drifts *worse* over time

> **Note**
>
> The dashed green line is the **least-squares** solution *on this exact noisy sample* — computed with the normal equation from a few slides back, not the "true" `(w, b)` we generated the data from. Because of the noise, those aren't the same thing, and least squares always wins on its own sample.

Comparing Mini-batch Sizes

Let’s vary minibatch size with 40 datapoints and a budget of 4000 evaluations



What turning the dial actually buys you

- Small k (light blue) is noisiest, but its many cheap steps still get it to a low loss fastest, early on
- Large k (dark blue) is the smoothest, but under a fixed budget it gets far fewer steps — and visibly lags behind here
- There’s no universally “best” k — it’s a real trade-off, tuned per problem, per budget, per hardware

The takeaway

> More steps with a noisy gradient often beats fewer steps with an exact one

- This is *why* SGD and mini-batch dominate in practice, especially on huge datasets — a single full-batch gradient descent step on a billion-row dataset costs a billion point-evaluations before `(w, b)` has moved even once
- Gradient descent isn't wrong — it's just not what anyone actually runs at scale
- For some concrete numbers on "huge dataset":
  - ImageNet is ~138GB
  - The Reddit dataset (used for graph learning benchmarks) is ~2.841TB